UNIVERSIDAD DISTRITAL FRANSISCO JOSÉ DE CALDAS

Faculty of Engineering

Project technical report

February 14, 2025

Authors:
Tito Burbano Plazas, 20231020169
Anderson Arenas Gutierrez , 20231020030

2024-2025

Contents

1 Introduction

The Trivia Game Management System is designed to facilitate the administration and execution of a question-and-answer game. This project supports comprehensive user management—including standard and administrator roles—question management, and dynamic game session handling. The overall system is architecturally divided into two primary components:

Java (Spring Boot) Component: This component is chiefly responsible for user management. It includes functionalities such as user registration, authentication (login and logout), and the creation of administrator accounts. The system persists user data in a JSON file (Users.json), ensuring that user information is stored in a structured, easily accessible format. The Java part follows a robust MVC (Model-View-Controller) architecture, enabling clear separation between data management, business logic, and presentation layers.

Python (FastAPI) Component: In parallel, the Python component is dedicated to the core game logic. It handles game session management, question retrieval and manipulation (using Questions.json), and exposes RESTful endpoints for operations like querying, filtering, and adding questions. FastAPI is employed to build a high-performance web API that is both developer-friendly and scalable. This component is organized into modules representing controllers (routers), services, and repositories, each with distinct responsibilities.

Both components rely on JSON files for data persistence and use environment variables (managed by the EnvironmentVariables class in Python and corresponding configuration properties in Java) to dynamically specify file paths and other sensitive configuration parameters. This dual-language architecture not only supports independent execution and scalability but also allows for potential future integration via RESTful services or shared data repositories.

---

2 User Stories

1. As a player, I want to choose a question category (history, science, entertainment, etc.) so I can answer questions on a topic that interests me.
2. As a player, I want to earn rewards for correctly answering questions, so I can stay motivated and achieve milestones within the game.
3. As a player, I want the game to have a ranking system, so I can compare my performance to other players.
4. As an admin, I want to be able to add questions, so I can keep the game updated and interesting.
5. As a player, I want to be able to log in to the game, so I can save my progress.
6. As a user, I want to search for all players, so I can see their information.
7. As a user, I want to search for a specific player, so I can view their information.
8. As an admin, I want to see all the questions in the game, so I can have that information readily available.
9. As an admin, I want to search for questions by a keyword, so I can check if it already exists in the game.
10. As a user, I want to be able to create an account in the game, so I can start playing.
11. As an admin, I want to be able to create an account in the game, so I can manage the game and its content.

12. As a user, I want to be able to log out, so I can stop playing and save my progress.

3 Functional and Non-Functional Requirements

3.1 Functional Requirements

3.1.1 User Management

Registration:
The system must allow new users to register, providing necessary details such as username, password, and an initial score. The registration process ensures that each user is uniquely identified and stored securely in the JSON file.

Authentication:
Users must be able to log in and log out securely. In the Java component, successful authentication updates the user's state (e.g., marking the user as "online") in the persisted JSON data. This facilitates subsequent authorization checks for protected operations.

Administrator Creation:
The system supports the creation of administrator accounts, but only if a current administrator is logged in. This safeguard ensures that elevated privileges are granted in a controlled manner.

User Information Retrieval:
The system provides endpoints and methods to retrieve user data by various parameters (such as ID or username). This is critical for both operational needs (e.g., viewing a leaderboard) and administrative tasks.

3.1.2 Question Management

Retrieval of Questions:
Users and administrators must be able to retrieve a complete list of available questions. The retrieval can be simple (fetching all questions) or refined (using filters).

Filtering:
The application must support filtering of questions by keywords or categories, enhancing the usability of the trivia game by allowing targeted queries.

Question Addition:
Administrators, once authenticated, have the authority to add new questions to the repository. This function ensures that the question pool can be dynamically updated to keep the game engaging.

3.1.3 Trivia Game Functionality

Session Initiation:
An online user should be able to initiate a game session seamlessly. The system checks the user's status and retrieves necessary data before beginning the game.

Gameplay Dynamics:
The game session involves presenting random questions to the user, processing their answers, and updating scores accordingly. The gameplay continues until the user exhausts their allotted lives, providing a clear challenge and progression.

Leaderboard Display:
After game sessions, the system compiles and displays a leaderboard. This ranking, based on user scores, encourages competitive play and ongoing engagement.

3.2 Non-Functional Requirements

3.2.1 Maintainability

The system's layered architecture (controllers/routers, services, repositories) ensures that modifications in one layer have minimal impact on others. This separation of concerns makes the codebase easier to understand, debug, and extend.

Dependency injection is utilized (via Spring Boot's `@Autowired` in Java and explicit repository instantiation in Python) to reduce tight coupling between classes, further enhancing maintainability.

3.2.2 Portability

Storing data in JSON files and using environment variables for file paths ensures that the system can be easily moved between different environments (development, staging, production) without major reconfiguration.

The independent operation of the Java and Python components supports flexible deployment scenarios, including microservices architectures.

3.2.3 Performance

JSON file-based persistence is efficient for small to moderate volumes of data, offering quick read/write operations. However, the system is designed to be scalable, with the understanding that a transition to a relational or NoSQL database may be necessary for higher data volumes.

3.2.4 Security

While the system currently implements basic login/logout functionalities, it stores passwords in plain text. This is a known security risk, and integrating a hashing mechanism (such as BCrypt) is recommended to enhance security.

Robust session management is needed, especially in scenarios with multiple concurrent users. Future improvements could include token-based authentication (e.g., JWT) to mitigate risks associated with simple state marking.

3.2.5 Usability

Clear and well-documented REST endpoints (using Swagger/OpenAPI specifications in both FastAPI and Spring Boot) ensure that external clients can easily integrate with the system.

User-friendly error handling and validation in both Java and Python components contribute to an intuitive interface for end-users.

4 Technical Considerations

4.1 Architecture

Java (Spring Boot):
The Java component follows the MVC design pattern. Controllers handle HTTP requests, services encapsulate the business logic, and repositories manage data persistence. This clear separation enables developers to work on distinct parts of the application without interfering with each other.

Python (FastAPI):
FastAPI leverages Python's dynamic capabilities to build high-performance APIs. Here, endpoints are defined

using APIRouter, while service classes encapsulate game logic and business rules. Repositories in Python are responsible for direct data manipulation (reading from and writing to JSON files).

4.2 Data Persistence

Data is stored in two main JSON files: `Users.json` for user data and `Questions.json` for question data. This simple persistence mechanism is well-suited for development and small-scale production environments.

The use of environment variables (managed by the `EnvironmentVariables` class in Python and configuration properties in Java) abstracts the file paths, making the system more portable and adaptable to different deployment environments.

4.3 Inter-Module Communication

The Java and Python components operate independently but are designed to interact through shared JSON file structures. In a future integration scenario, these components might also communicate via RESTful APIs.

This decoupling means that each component can evolve independently, as long as the shared data format (the structure of the JSON files) is maintained.

4.4 Session Management and User State

In the Java component, user sessions are managed by updating the user's state in the JSON file (e.g., marking a user as "online" upon login). While this method is straightforward, it may not scale well in high-load scenarios.

The Python component's game session (`GameSession_Py`) first verifies that the user is online before initiating gameplay, ensuring that only authenticated users can participate.

4.5 Environment and Configuration

The use of environment variables is a critical technical consideration. The `EnvironmentVariables` class in Python (and the equivalent configuration mechanisms in Java) ensures that sensitive data such as file paths are not hard-coded. This design enhances security and makes the application more adaptable across various deployment settings.

5 UML Diagrams

To see the uml disgrams please go to the folder called "Documentation", unfortunately an error occurs when loading the images (as you can see in point number six, the images are directed to another uncorresponded section, but we decided to change it because with more than 24 images it looks very bad, in point six there are only 3)

6 Architecture

7 Analysis of SOLID Principles in the Architecture

7.1 Single Responsibility Principle (SRP)

Each class in the project has a clearly defined responsibility:

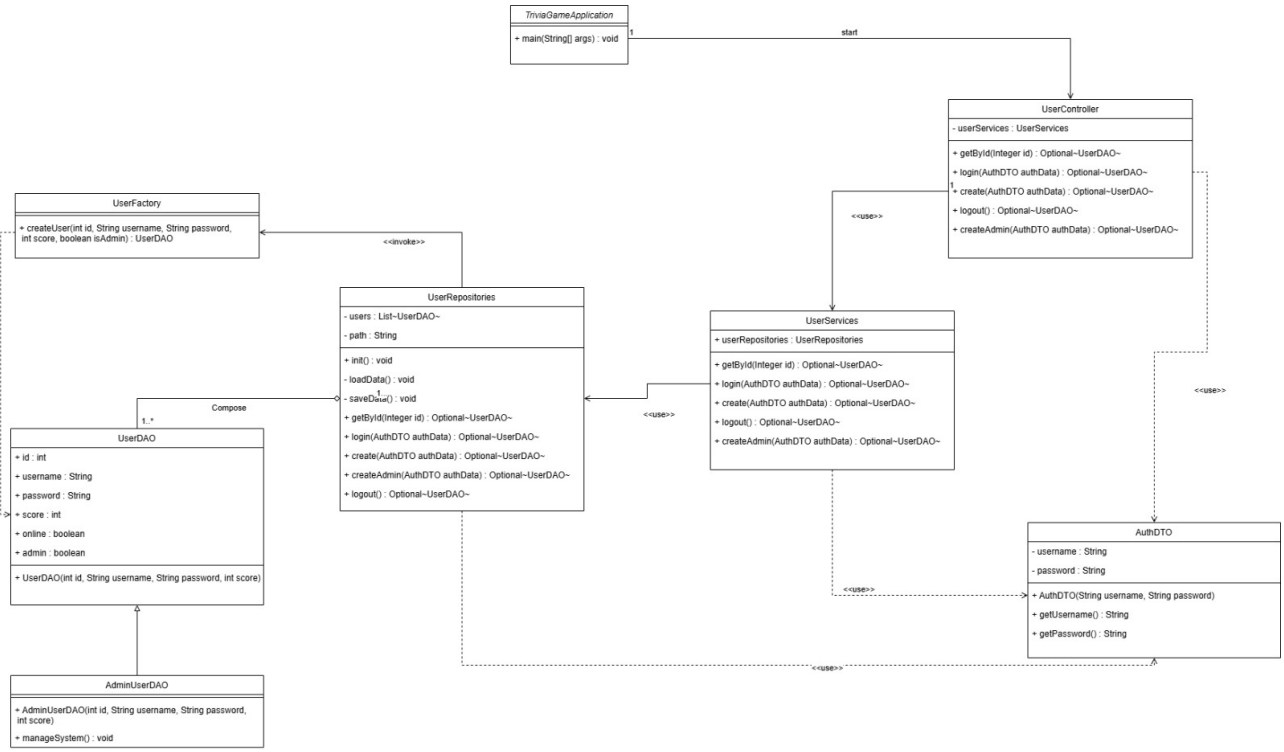

Figure 1: Java Class Diagram

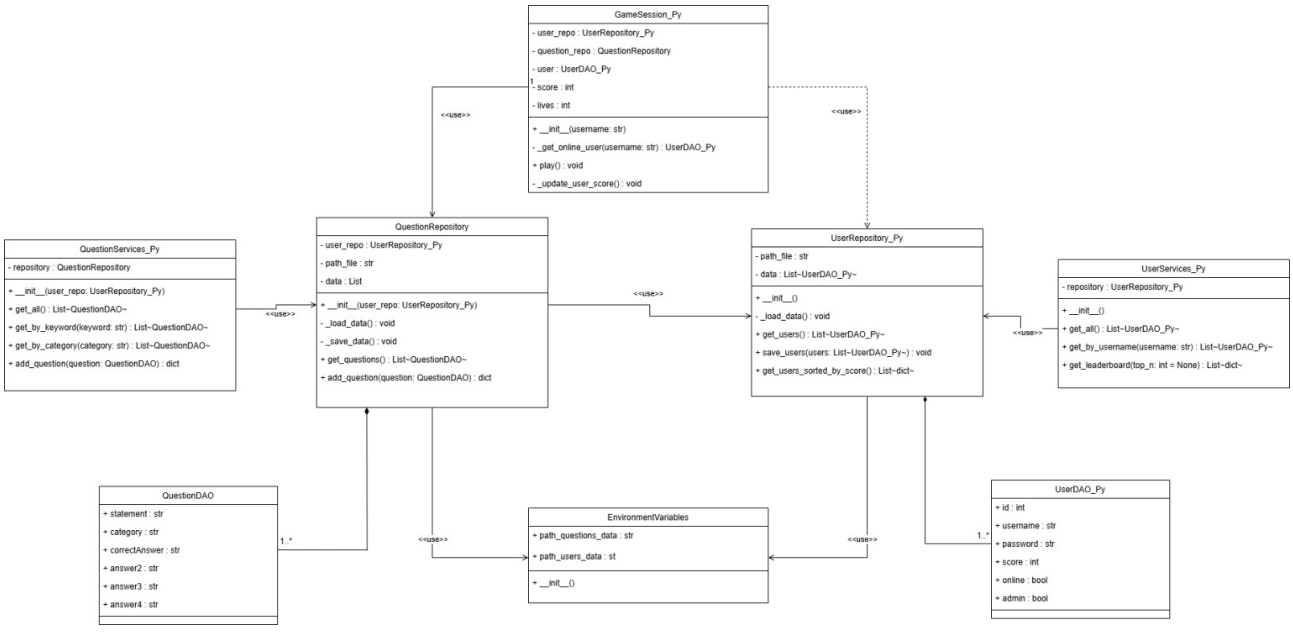

Figure 2: Python Class Diagram

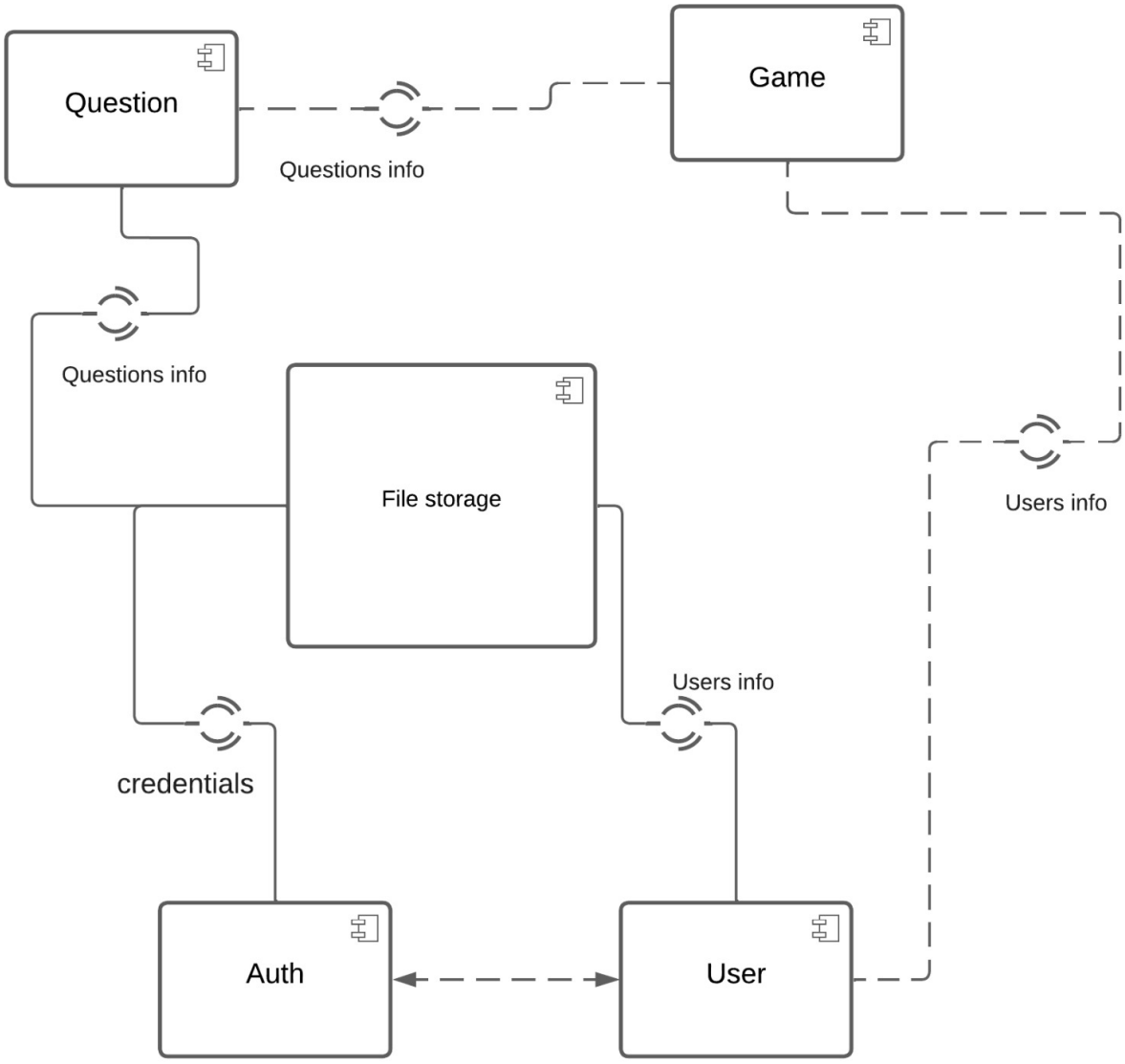

Figure 3: Component Diagram

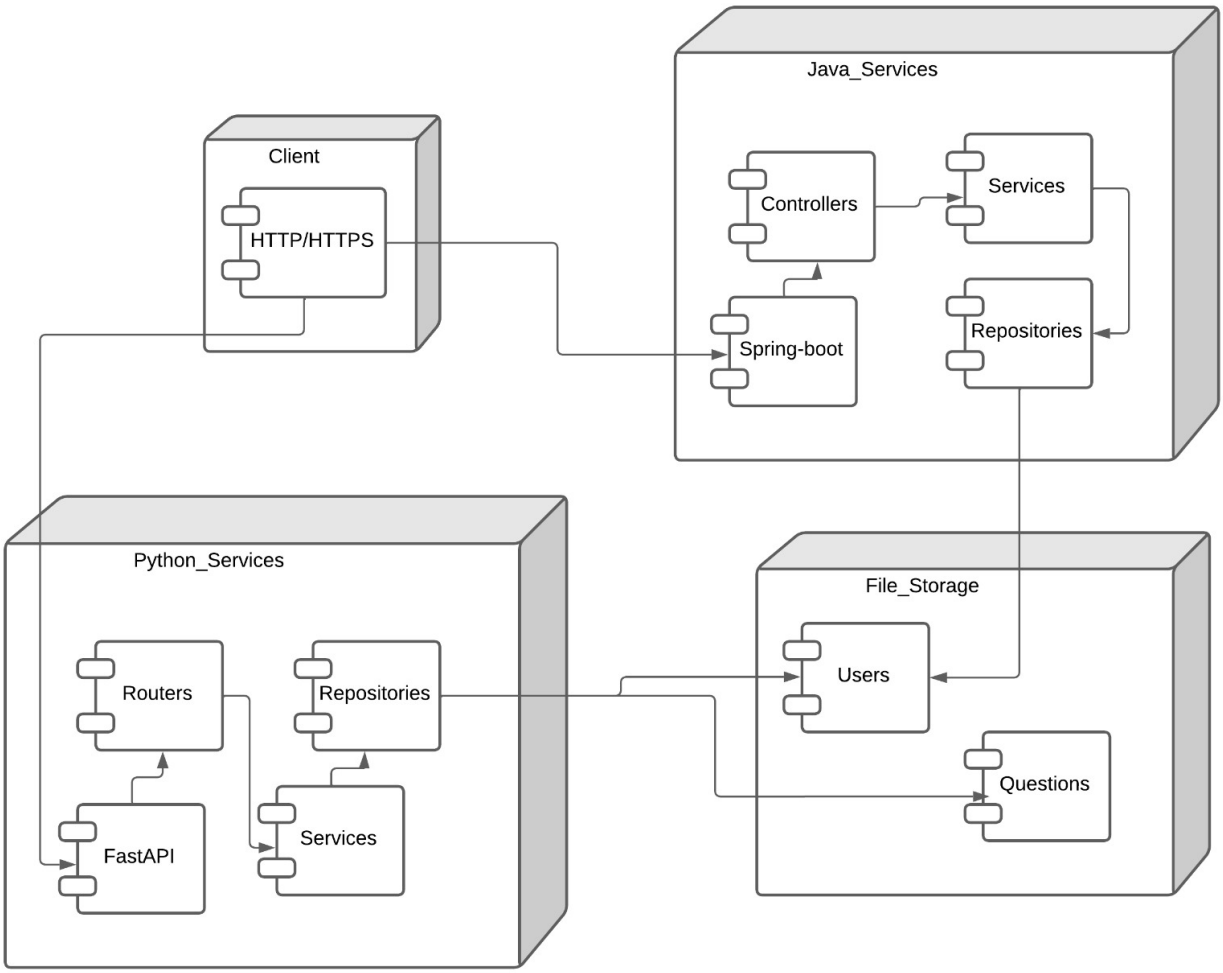

Figure 4: Deployment Diagram

- Controllers/routers handle HTTP requests and route them to appropriate services.
- Services encapsulate the business logic, such as user authentication, game session management, and question filtering.
- Repositories are solely responsible for data persistence, reading from, and writing to JSON files.
- Factories (like `UserFactory`) manage object creation, isolating instantiation logic from the rest of the system.

This strict adherence to SRP not only simplifies maintenance but also enhances the testability of individual components.

7.2 Open/Closed Principle (OCP)

The system is designed to be open for extension but closed for modification:

- For instance, the `UserFactory` can be extended to handle new user types (e.g., a moderator role) without altering existing service or repository code.
- The modular nature of the services and repositories means that new functionalities (such as additional filters or new business rules) can be added by extending existing classes rather than modifying them.

7.3 Liskov Substitution Principle (LSP)

In this project:

- `AdminUserDAO` is a subclass of `UserDAO` in the Java component, ensuring that any instance of `AdminUserDAO` can be used wherever a `UserDAO` is expected without introducing errors or unexpected behavior.
- In Python, although explicit subclassing of DAOs is not employed, the design maintains consistency by using uniform data models (e.g., `UserDAO_Py`), ensuring that similar operations can be performed on all user objects.

7.4 Interface Segregation Principle (ISP)

While the project does not employ explicit interface declarations in Java or abstract base classes in Python, each module exposes only the methods necessary for its functionality:

- Controllers provide only the endpoints relevant to user and question management.
- Services offer methods specifically designed for the business logic they encapsulate.

This design minimizes dependencies on unnecessary methods and keeps the interfaces clean and focused.

7.5 Dependency Inversion Principle (DIP)

The system effectively decouples high-level modules from low-level details:

- **Java:** The use of Spring Boot's dependency injection (`@Autowired`) allows controllers to rely on service interfaces rather than concrete implementations.

- **Python:** Although a formal dependency injection framework is not used, repositories are passed to service classes upon instantiation. This inversion ensures that services depend on abstractions (the repository interface) rather than concrete file-handling implementations, facilitating easier testing and future modifications.

8 Analysis and Detailed Implementation of Design Patterns

In this project, three key design patterns have been effectively implemented. The following sections detail the role, implementation, and benefits of each pattern.

8.1 Factory Pattern

8.1.1 Description and Objective

The Factory Pattern is a creational design pattern that encapsulates the logic for object creation. Its main objective is to centralize and abstract the instantiation process, so that client code does not need to know the details of the object's concrete implementation. This leads to a more modular and maintainable system.

8.1.2 Implementation in the Project (Java Context)

Class: UserFactory

How It Works: When the system needs to create a new user, instead of directly instantiating either `UserDAO` or `AdminUserDAO`, the client code calls `UserFactory.createUser(...)`, where the factory method checks the `isAdmin` parameter and returns an instance of the appropriate class.

Code Example (Simplified):

```
public class UserFactory {
    public static UserDAO createUser(int id, String username, String password, int score, boolea
        if (isAdmin) {
            return new AdminUserDAO(id, username, password, score);
        }
        return new UserDAO(id, username, password, score);
    }
}
```

8.1.3 Benefits and Advantages

- **Centralized Creation Logic:** All object creation logic is isolated within `UserFactory`. This means that any changes required (such as additional validation, logging, or initialization of new attributes) are confined to this class.
- **Enhanced Flexibility:** The system can easily incorporate new types of users (e.g., moderators or premium users) without modifying the code in controllers or services. New conditions can be added to the factory method to decide which concrete class to instantiate.
- **Reduced Coupling:** Client classes (like repositories or services) no longer depend on concrete implementations. They interact with a common interface (`UserDAO`), which simplifies testing and future modifications.

8.2 Facade Pattern

8.2.1 Description and Objective

The Facade Pattern provides a unified, simplified interface to a complex subsystem. By abstracting the underlying complexities, it allows clients to interact with the system without needing to understand its internal workings. This is especially beneficial in systems with multiple interdependent components.

8.2.2 Implementation in the Project (Java and Python Context)

Service Classes as Facades: Both the Java and Python layers use service classes as facades. For example:

Java: The `UserServices` class acts as an intermediary between the `UserController` and `UserRepositories`. It encapsulates the business logic, performing necessary validations and orchestrating calls to the repository. The controller only needs to call simple methods like `login` or `create`, unaware of the underlying data access complexities.

Python: Similarly, `QuestionServices_Py` provides methods such as `get_all()`, `get_by_keyword()`, and `add_question()`. These methods abstract away the file operations and any intricate logic related to filtering or validation. Endpoints defined in FastAPI simply interact with these service methods.

8.2.3 Benefits

- **Simplification of Client Interactions:** Clients (controllers/routers) receive a clear and concise interface to perform operations. They are shielded from the complex interactions among various components, leading to simpler and more maintainable code.
- **Decoupling:** Changes in the underlying data access or business logic do not affect the client's code as long as the service interface remains unchanged. This makes the system more resilient to changes.
- **Enhanced Maintainability and Scalability:** When new functionality or modifications are required, developers can update the service layer without affecting the presentation layer. This separation improves both maintainability and scalability over time.

8.3 Composite Pattern in Repositories

8.3.1 Description and Objective

The Composite Pattern is a structural design pattern that allows you to treat individual objects and compositions of objects uniformly. It is used to build tree structures where clients can interact with both single objects (leaves) and groups of objects (composites) through the same interface. In this project, we adapt the principles of the Composite Pattern to manage collections of Data Access Objects (DAOs) within repositories.

8.3.2 Implementation in the Project

Java Context:

Class: UserRepositories

Functionality: `UserRepositories` maintains a collection (e.g., a list) of `UserDAO` instances. This structure can be thought of as a composite where the repository represents the whole, and each individual `UserDAO` is a leaf. Although the system does not implement a full hierarchical tree (as seen in typical composite pattern examples), the approach still allows the repository to uniformly manage operations such as adding, removing, or querying user objects.

Python Context:

Classes: `UserRepository_Py` and `QuestionRepository`

Functionality: Both repositories in Python manage a collection of DAOs (`UserDAO_Py` for users and `QuestionDAO` for questions). The repositories provide methods to retrieve, add, or update the entire collection. This unified treatment of individual data items and the collection as a whole aligns with the composite pattern's intent.

8.3.3 Benefits

- **Uniformity in Operations:** The composite structure allows for common operations (such as iterating over the collection, performing bulk updates, or persisting changes) to be applied uniformly, regardless of the number of DAO instances.
- **Centralized Data Management:** By grouping DAOs within a composite repository, the system centralizes data management, making it easier to enforce consistency, perform validations, and maintain the lifecycle of the data.
- **Enhanced Modularity and Reusability:** Each repository is responsible for a specific entity type. The composite approach makes it straightforward to extend or modify the repository's behavior (for example, changing the way data is filtered or sorted) without impacting other system parts.
- **Clear Dependency Relationships:** The explicit modeling of the repository-DAO relationship reinforces the repository's role as the sole manager of its contained objects. This clarity aids in debugging and future modifications.

9 Analysis of Programming Best Practices and Anti-Patterns

9.1 Best Practices

- **Layered Architecture:** The project is structured into distinct layers: controllers (or routers), services, and repositories. This separation of concerns minimizes interdependencies and makes the system easier to understand, maintain, and test.
- **Comprehensive Documentation:** Detailed documentation is provided across the codebase—Java components use JavaDoc while Python modules include extensive docstrings. This clarity ensures that developers can quickly understand the purpose and behavior of each class and method.
- **Dependency Injection:** The use of dependency injection in Java (via Spring Boot's `@Autowired`) and the explicit instantiation of repositories in Python help decouple high-level modules from low-level implementation details. This promotes easier unit testing and better modularity.
- **Use of Data Models:** Data models such as `UserDAO`, `AdminUserDAO`, and `QuestionDAO` (and their Python counterparts) are clearly defined to represent the domain entities. This practice ensures data integrity and facilitates validation through tools like Pydantic in Python.
- **Environment Configuration:** The system avoids hardcoding sensitive data by using environment variables (handled by the `EnvironmentVariables` class in Python and configuration properties in Java). This enhances security and portability.

9.2 Anti-Patterns or Risks

- **Plain-Text Password Storage:** Currently, passwords are stored in plain text within the JSON files. This represents a significant security vulnerability. Implementing a robust hashing mechanism (such as BCrypt) is essential to mitigate this risk.
- **Basic Session Management:** The method of marking a user as "online" directly in the JSON file is a simplistic approach to session management. This can lead to scalability issues and potential security flaws in multi-user environments. Transitioning to token-based authentication (e.g., JWT) or a dedicated session management system would provide a more secure and scalable solution.
- **Reliance on JSON for Persistence:** While JSON files are suitable for small datasets, they can become a performance bottleneck as data volume increases. Migrating to a relational or NoSQL database would enhance scalability, performance, and reliability.
- **Duplicated Logic Across Components:** The existence of similar user and question models in both the Java and Python parts (e.g., `UserDAO` vs. `UserDAO_Py`) could lead to inconsistencies if both implementations are not carefully synchronized. Consolidating or abstracting common logic could reduce maintenance overhead and potential errors.

10 Conclusions

10.1 Successful Multi-Language Integration

The project demonstrates a successful integration of two independent components (Java and Python) that work together to manage a trivia game system. Despite the inherent challenges in maintaining consistency across different programming languages, the system's reliance on shared JSON data structures facilitates smooth operation.

10.2 Effective Use of Design Patterns

The application of the Factory, Facade, and Composite patterns significantly enhances the system's modularity, flexibility, and maintainability. The Factory Pattern centralizes object creation, the Facade Pattern simplifies interactions with complex subsystems, and the Composite Pattern allows for uniform management of data collections.

10.3 Adherence to SOLID Principles

The architectural design shows strong adherence to SOLID principles. Each class is dedicated to a specific responsibility, classes are open for extension, and dependencies are managed through inversion of control. This disciplined approach makes the system robust and adaptable to future changes.

10.4 Areas for Improvement

Several opportunities exist to further strengthen the system:

- **Security Enhancements:** Implement password hashing and more robust session management (e.g., JWT).
- **Scalability Enhancements:** Consider transitioning from JSON-based persistence to a full-fledged database solution for handling larger datasets.

- **Code Consolidation:** Reduce duplication between the Java and Python implementations to prevent inconsistencies and simplify maintenance.

10.5 Overall Impact

The well-structured, layered architecture coupled with thoughtful design patterns and adherence to programming best practices results in a system that is both maintainable and scalable. The project effectively meets its functional and non-functional requirements, positioning it well for future enhancements and broader deployment scenarios.